

Project Proposal: skyjam, Forecasting GNSS Interference over European Airspace

Zayden (github.com/Zayden16) · MLOps HS26 · MS1 Proposal
Repository (public): <https://github.com/Zayden16/hslu-mlops-hs-26>

1 Problem statement

What and for whom. GNSS jamming and spoofing are now routine over the Baltic, the Black Sea and the Eastern Mediterranean: aircraft lose the integrity of their satellite position fix mid-flight. Public tools such as gpsjam.org show *yesterday's* map. A dispatcher planning a route, a drone operator or a flight-ops centre needs the opposite: a statement about the hours ahead.

Prediction. For every H3 resolution-2 airspace cell (~180 km edge), predict the probability that the cell is *interference-affected exactly 6 hours ahead*. This is binary classification per cell-hour, re-issued every hour. **Scope:** European and adjacent airspace, covered by 18 sampling discs of 250 NM (11 over regions with reported interference, 7 over Western-European control regions), about 80 populated cells per sweep.

Label. A cell-hour is affected when at least **30 %** of the cruise-altitude aircraft in it broadcast a degraded navigation integrity category (NIC < 7, containment radius worse than 0.2 NM). NIC is each aircraft's own assessment of its position quality, sent over ADS-B. The label is a *fleet-level consensus about a physical event* at a future timestamp, aggregated over independent airframes, not a rule applied to the model's inputs.

Success criterion. On a strictly out-of-time test split the model must beat two baselines:

Baseline	Why it is the bar
Persistence: state at t repeated at $t + 6$ h	Interference is strongly autocorrelated; not beating this means nothing was learned
Per-cell hourly climatology	Encodes “the Gulf of Finland is usually bad at 14:00” with no live signal

Target: PR-AUC at least 0.10 above persistence and a lower Brier score, with the gain holding *separately* in interference and control regions. **Rare positive class** (4.5 % of cells in the first validation snapshot): accuracy is meaningless, so PR-AUC is the headline metric. It is handled with class weighting and a decision threshold tuned on the validation split, and precision/recall are reported per region, because a model that permanently flags the whole Baltic would look good globally and be operationally useless.

2 Originality & motivation

Checked against the HSLU archive (mlops-lab.ch) and the KTH ID2223 project lists: both are dominated by crypto prices, weather, air quality and transport availability. None predicts the *quality* of a navigation signal, and no ADS-B project appears. **In one sentence: the dataset does not exist until my pipeline creates it, and the label is inferred from fleet telemetry rather than read from a column.**

- No historical download exists** for per-cell navigation integrity. The feature pipeline is the sole source of history, so the scheduled job is load-bearing, not decorative.
- The label is constructed** from a fleet-level statistic over self-reported avionics quality, with a defensible threshold and an explicit traffic floor.
- Drift is the subject, not an afterthought.** A new emitter switching on is genuine concept drift in its region, so MS4 monitoring measures the very phenomenon being modelled.

Motivation: GNSS interference is an active civil-aviation safety concern in Europe, the raw telemetry is fully public, and I want a project whose failure modes are physical rather than financial.

3 Data source & features

Source. `api.adsb.lol` v2, a public community ADS-B aggregator, endpoint `/v2/point/{lat}/{lon}/{radius}`. A documented JSON API, **not scraping**, free and key-less for non-commercial use. **Fallback:** `api.adsb.fi` serves the same schema and is a one-variable switch (`SKYJAM_ADSB_BASE_URL`).

Update frequency and volume. The feed is real time. The pipeline sweeps all 18 discs every **30 minutes**, pacing requests 12 s apart (the measured rate limit) with exponential backoff. History **starts when my poller starts**; there is no back-download. As of 26.09.2026 the store holds **1.09 M observations over 189 hourly slots** and grows by roughly 140 k observations, or about 1 900 labelled cell-hours, per day, so MS3 will train on the order of 100 000 cell-hours.

Validated signal (live snapshot, aircraft at or above FL200, degraded = NIC < 7):

Interference region	Deg.	Interference region	Deg.	Control region	Deg.
Gulf of Finland	52.0 %	Baltic south	13.2 %	Germany	2.4 %
Poland east	34.0 %	Romania/Moldova	6.2 %	Switzerland	1.4 %
Kaliningrad	25.9 %	Black Sea west	5.1 %	UK / France	1.0 / 0.0 %

A roughly 40-fold separation between worst region and controls: the target is real, not assumed. **The altitude filter matters:** unfiltered, the Swiss disc shows 8.9 % degraded aircraft against 2.3 % at FL200+, because cheap general-aviation avionics report poor integrity everywhere; the same filter leaves the Baltic figures essentially unchanged.

Named features per cell and hourly slot: `aircraft_count`, `degraded_share`, `median_nic`, `p25_nic`, `mean_nac_p`; lags of `degraded_share` and of the binary state at 1, 2, 3, 6, 12, 24 and 168 h; backward-looking rolling means of `degraded_share` over 6 h and 24 h; `aircraft_count_mean_24h`; calendar features `hour_utc`, `day_of_week`.

Leakage. (i) *Split*: strictly chronological train / validation / test with a 6-hour gap between segments so no example straddles a boundary; no shuffling anywhere. (ii) *Temporal, not positional lags*: each cell is reindexed onto a gap-free hourly grid first, because a positional `shift(1)` would present a three-hour-old value as one hour old whenever a sweep was missed; a unit test asserts the lag across a removed hour is NaN. (iii) *Rolling windows exclude the present* (shifted one slot, tested). (iv) *Unobserved is not quiet*: cells under five aircraft are dropped, not imputed as clean, and rows whose future state was never observed carry no label.

4 System design

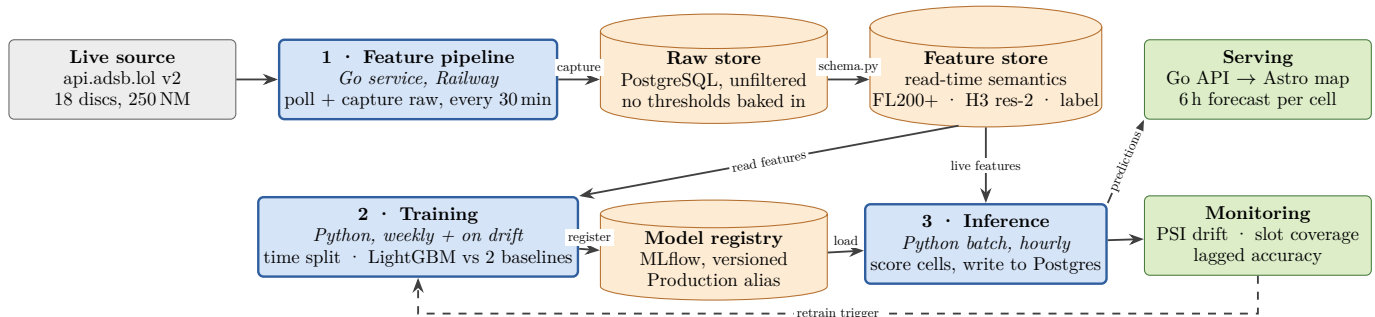


Figure 1: FTI architecture. Blue: the three pipelines; orange: stores; solid arrows: data flow; dashed: retrain trigger. Deployed today: the live source, feature pipeline and raw store.

Pipelines and triggers. (1) *Feature*: a Go service on Railway sweeps the 18 discs every 30 minutes and writes **unfiltered** observations to PostgreSQL: raw coordinates, nullable altitude, no H3 cell and no label. Thresholds are applied at *read* time, so the altitude floor or H3 resolution can change and the whole history be rebuilt. (2) *Training* (Python): weekly and on drift; reads the store, splits chronologically, trains LightGBM against both baselines, logs to MLflow, registers the best run. (3) *Inference* (Python): an hourly batch job, run after each sweep, that loads the Production model and writes a 6-hour forecast per cell to a PostgreSQL table. A Go API serves that table to an Astro frontend, so the request path never touches Python.

Tech stack, one line of rationale each:

- **Languages**: Go wherever a process runs continuously in production (capture, serving API); Python only where the ML ecosystem is (features, LightGBM, MLflow). Measured on the deployed capture service: a 21 MB distroless image against 665 MB for the Python one, 16 MB resident memory, one static binary with no runtime to patch. Its only dependency is the Postgres driver, and its concurrency and timeouts come from the standard library.
- **Cloud and feature store**: Railway (container hosting) + PostgreSQL. Durable, unlike the 90-day CI artifacts it replaced, and SQL applies thresholds at read time.
- **Orchestration**: a long-lived Go service with a wall-clock ticker. Four staggered GitHub Actions crons captured only 53 % of hours over 74 h (GitHub drops scheduled triggers and never retries), and here a missed hour is lost forever.
- **Tracking and registry**: MLflow, for run comparison and a Production alias that inference loads, so the deployed version is always identifiable.
- **Serving**: a Go JSON API plus an Astro frontend with an H3 map, both on Railway; static-first pages with a map island, and a map is the natural view of a spatial forecast.
- **Packaging and CI**: Docker, `uv.lock` for Python, Go modules; GitHub Actions runs lint and 50 tests (34 Python, 16 Go) against a real PostgreSQL.

Training-serving skew is prevented structurally: thresholds, altitude floor, H3 resolution and lag set live once in `src/skyjam/common/schema.py` and are applied on read. The Go capture service computes **no** label, and the sampling grid it shares with Python has a parity test that fails the build if the two drift.

Optional / stretch (not core): Terraform IaC, Airflow, a managed feature store, alerting on the drift metric. Core FTI comes first.

Current state and risks. The capture service is deployed and has completed 18/18 discs on every recent sweep. Risks: history accrues only from now on (mitigated by capturing since proposal time and storing raw, so a definition change does not invalidate it); the API could throttle or disappear (measured pacing, backoff, `adsb.fi` fallback); sparse night traffic (per-cell traffic floor, unobserved cells excluded). **The repository is public**: <https://github.com/Zayden16/hslu-mlops-hs-26>.